LAMP: ASP Rule Induction and Optimization with LLMs

David Bunker
University of Potsdam

Abstract

Answer Set Programming (ASP) offers a compelling mechanism for knowledge representation and reasoning, however developing such programs remains challenging. This work introduces the LLM to ASP Modeling Protocol (LAMP), a system by which an LLM iteratively generates and refines candidate ASP rules with each candidate verified by the Clingo solver. LAMP is evaluated against the leading Inductive Logic Programming (ILP) system Inductive Learning of Answer Set Programs (ILASP), across 120 synthetic benchmarks of varying complexity. ILASP achieves 97.5% accuracy, with failures due to search space timeouts. Among LLMs, *gpt-5-mini* reaches 100% accuracy, *gpt-oss:20b* 95.8%, and *qwen3-coder:30b* 45.0%. Although ILASP was on average faster, the leading LLM solved all benchmarks on which ILASP timed out. Both LAMP and ILASP are also able to reduce literal counts and increase stratification at a similar level. Finally, feature analysis is used to determine predictors of LLM success or failure.

1 Introduction

Knowledge representation and reasoning problems arise across many domains, including software and hardware customization, industrial manufacturing, service composition and others. These problems are characterized by combinatorial constraints that determine which combinations of components are valid. Answer Set Programming (ASP) models such problems, offering clear semantics with expressive capabilities for constraints, defaults, and combinatorial reasoning.

However, writing ASP programs that are both correct and performant remains challenging, as modeling choices can significantly affect solver behavior. Recent progress in Large Language Models (LLMs) offers new opportunities to reduce this modeling burden. LLMs have shown strong capabilities in code generation and program synthesis, suggesting they may assist in deriving ASP encodings from high level natural language specifications [15].

The LLM to ASP Modeling Protocol (LAMP) is designed to guide LLMs in deducing and optimizing ASP rules from given instance facts and expected

answer sets. LAMP provides a structured interaction protocol that enables refinement of ASP encodings, with the goal of improving both correctness and performance. Beyond proposing the system itself, LAMP can be used as a framework for evaluating the capabilities of different LLMs in ASP program generation.

Both LAMP and the classical Inductive Logic Programming (ILP) system Inductive Learning of Answer Set Programs (ILASP) [20] address the same underlying problem class. Given a set of instance facts and an expected answer set, find a program within a bounded hypothesis space that produces the correct answer sets. ILASP solves this via exhaustive search under the Learning from Answer Sets (LAS) framework while LAMP replaces exhaustive search with LLM guided generation and Clingo based iterative verification. As the expected input and output are the same a controlled comparison can be made between symbolic ILP search and LLM based generation on the same class of tasks.

The relevant research questions of this work are the following.

RQ1. Is a structured approach for LLM assisted ASP modeling possible using the LAMP system and if so under what conditions do LLMs outperform, match, or fall short of classical ILP ILASP on ASP rule induction?

RQ2. How do LLMs and ILASP compare in terms of accuracy, speed, and program conciseness on ASP rule induction tasks?

RQ3. Which features of the ASP benchmark most strongly predict LLM success or failure in LAMP?

This paper investigates the current capabilities of Large Language Models (LLMs) regarding Answer Set Programming (ASP) and offers directions for future research at the intersection of artificial intelligence and constraint solving. The LAMP system can be compared against similar systems such as those based on LLMs fine-tuned to ASP [6], iterative scheduling and plan generation with LLMs and ASP [22], and LLM based systems that can learn directly from ASP answer sets [4], as well as solver-in-the-loop frameworks that use Clingo feedback to iteratively improve LLM generated ASP [26].

2 Background

This section presents the core background underpinning LAMP including Answer Set Programming (ASP), Inductive Learning of Answer Set Programs (ILASP), and Large Language Models (LLMs).

2.1 Answer Set Programming

Answer Set Programming (ASP) is a declarative programming paradigm based on stable model semantics [3].

Syntactic Primitives. A *term* is either a constant (a lowercase symbol or integer such as a, 0), a variable (an uppercase symbol, such as X), or a function symbol $f(t_1, \ldots, t_k)$ applied to terms. A *predicate* p/k is a symbol of arity $k \geq 0$. Applying it to k terms yields an *atom* $p(t_1, \ldots, t_k)$. A ground atom contains no variables. A *literal* is either a positive literal a (an atom) or a *default-negated* literal *not* a, where *not* denotes negation as failure and *not* a holds whenever a cannot be derived.

Rules and Programs. An ASP program Π is a finite set of *rules* of the following form.

$$h \leftarrow b_1, \ldots, b_m, \mathit{not}\ c_1, \ldots, \mathit{not}\ c_n \tag{1}$$

Here h is the *head* (a classical atom or empty), each b_i is a positive body literal, and *not* c_j denotes default negation of atom c_j. When $m = n = 0$ the rule is a *fact* and when h is empty the rule is a *constraint* (written $\leftarrow$ body), eliminating any answer set in which the body is satisfied.

Variables and Grounding. Rules are written with first order variables, allowing compact expression over large domains. Prior to solving, a *grounder*, such as GRINGO, substitutes all domain consistent constants for each variable, producing an equivalent propositional ground program Π_g. A rule is *safe* if every variable in the head or in a negative body literal also appears in a positive body literal, which guarantees that grounding is finite and well defined.

Semantics. Let $\mathcal{A}$ denote the Herbrand base set of all ground atoms. Given a ground program Π and a set $S \subseteq \mathcal{A}$, the *Gelfond–Lifschitz reduct* Π^S is obtained by the steps.

1. Remove every rule whose body contains *not* c for some $c \in S$
2. Remove all negative literals from the bodies of remaining rules.

S is a *stable model*, referred to in ASP as an *answer set*, of Π if and only if S is the minimal model of the reduct Π^S. A program may have zero, one, or multiple answer sets and solutions to the encoded problem correspond exactly to its answer sets.

Modern ASP solvers, such as Clingo [12], extend the core language with aggregates, choice rules, disjunctive heads, and optimization statements. LAMP uses Clingo as its verification oracle but operates over a restricted subset of normal rules only, as defined in Section 3.1.

Solving Complexity. Deciding whether a ground ASP program has an answer set is Σ_2^P-complete [2]. This places ASP one level above NP in the polynomial hierarchy, reflecting the two sources of computational hardness, the nondeterministic search over candidate sets $S \subseteq \mathcal{A}$, and the co-NP check that no proper subset of S is also a stable model of Π^S.

2.2 Learning from Answer Sets

Inductive Learning of Answer Set Programs (ILASP) [20] is a framework for Inductive Logic Programming (ILP) that learns ASP programs from examples. ILASP operates over three inputs. These are a *hypothesis space* $\mathcal{H}$, a *background knowledge* program B, and a set of *learning examples* E.

Hypothesis Space. The hypothesis space $\mathcal{H}$ is defined by a *mode bias*, which constrains the structure of rules that may appear in a learned hypothesis. A *mode declaration* is either a *head declaration* $modeh(a)$ or a *body declaration* $modeb(a)$, specifying which atoms may appear in rule heads and bodies respectively. Each atom in a mode declaration may contain *placeholder terms* of the form $\#var(t)$ (a variable of type t) or $\#con(t)$ (a constant of type t), which are instantiated during the search. A *hypothesis* $H \in \mathcal{H}$ is a set of ASP rules consistent with the mode bias.

Learning Examples. ILASP supports three types of examples, each placing a different constraint on the answer sets of $B \cup H$:

- **Positive examples** $e^+ \in E^+$: at least one answer set of $B \cup H$ must satisfy e^+.
- **Negative examples** $e^- \in E^-$: no answer set of $B \cup H$ may satisfy e^-.
- **Context-dependent examples** $e^{cd} \in E^{cd}$: a pair $\langle e_c, e_p \rangle$ where e_c is a *context* (an additional ASP program) and e_p is a *partial interpretation*. At least one answer set of $B \cup H \cup e_c$ must extend e_p, meaning it must include all atoms in e_p^+ and exclude all atoms in e_p^-.

Context-dependent examples subsume positive and negative examples and are the primary example type in modern ILASP, enabling the expression of partial observations under specific conditions.

Learning Task. A *learning task* is a tuple $T = \langle B, \mathcal{H}, E^+, E^-, E^{cd} \rangle$. A hypothesis $H \in \mathcal{H}$ is an *inductive solution* of T if it covers all examples. Every positive and context-dependent example is satisfied by some answer set of $B \cup H \cup e_c$, and no answer set of $B \cup H$ satisfies any negative example. When hypotheses are assigned a *cost* (typically the sum of rule lengths), the

learning task seeks an *optimal* solution. This is the inductive solution of minimum cost.

Search Procedure. ILASP reduces the learning task to a sequence of ASP solving problems. The hypothesis space is encoded as an ASP metaprogram, and the search for a covering hypothesis is directed by a *conflict-driven* procedure [20]. Candidate hypotheses are proposed, checked against all examples, and conflicts, examples not yet covered, are used to prune the space and guide subsequent iterations. This conflict-driven approach, implemented in the FastLAS system, scales substantially better than exhaustive enumeration of $\mathcal{H}$ [19].

Solving Complexity. Learning an optimal inductive solution under the full ILASP framework is Σ_3^P-complete [1], one level above the Σ_2^P-hardness of ASP itself. The additional level reflects the search over hypotheses $H \in \mathcal{H}$ layered on top of the Σ_2^P check that H is a valid inductive solution. This theoretical hardness motivates an alternative approach, rather than exhaustive symbolic search, LAMP replaces the hypothesis search with LLM-guided generation, bypassing the Σ_3^P search.

2.3 Large Language Models for Code Generation

Large Language Models (LLMs) are neural language models trained on large amounts of text and source code, enabling strong performance on code generation tasks. Given a structured prompt, an LLM generates a candidate program token by token. Its output is non-deterministic and may require post-processing or validation against a formal oracle.

LLMs have been shown to generate logic programs from natural language specifications [15] and to perform inductive logic programming when paired with a formal inference engine [10]. A key pattern across these systems is *iterative refinement*: the LLM generates a candidate, the candidate is verified symbolically, and any mismatch is encoded as feedback for the next prompt. LAMP instantiates this pattern for ASP, using Clingo as the verification oracle and Clingo-derived feedback to guide successive generation attempts.

3 Methodology

The LAMP framework operates on a formally defined benchmark structure that constrains the hypothesis space and the target answer set, as illustrated in Figure 1. This section defines that benchmark framework, describes how benchmark instances are generated, and then details the LAMP interaction protocol and evaluation criteria.

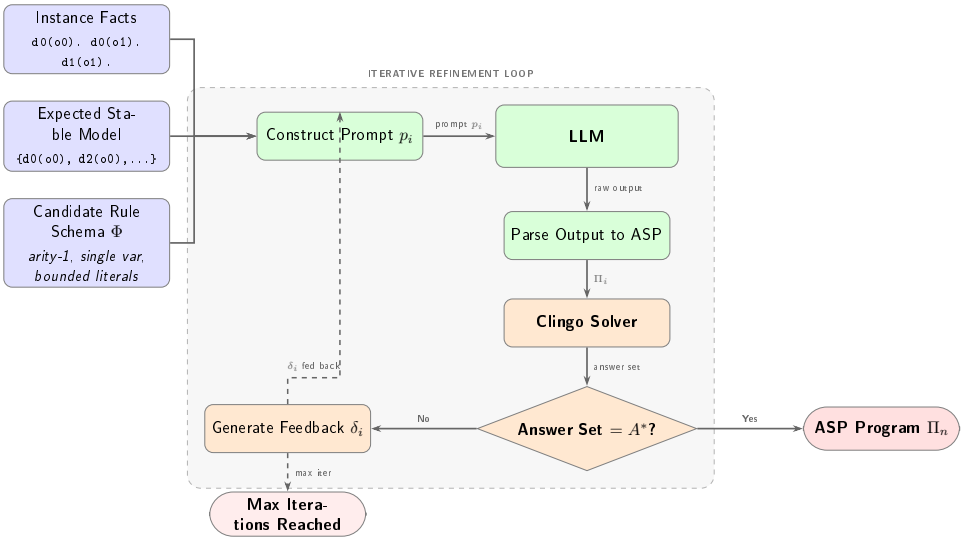

Figure 1: Overview of the LAMP system architecture.

3.1 Benchmark Framework

To keep the benchmarks tractable for both LLM generation and ILP search, the ASP used here is deliberately restricted. Programs consist only of unary *facts* of the form `descriptor(object)` and *normal rules* of the form `new_descriptor(X) :- descriptor_0(X), [not] descriptor_1(X), ...` which derive new object descriptions from initial instance facts. This subset is expressive enough to require fairly complex rule induction while remaining within a reasonable hypothesis space.

The following is an example representing the model facts that objects `"car"` and `"scooter"` are both vehicles, represented with the predicate `vehicle`, and the scooter is slow moving, represented with the predicate `slow_moving`. It also has the rule that if an object is a vehicle and not slow moving it is highway possible, represented with the predicate `highway_possible`.

Listing 1: Vehicle example ASP program.

```
vehicle("scooter").
vehicle("car").
slow_moving("scooter").
highway_possible(X) :- vehicle(X), not slow_moving(X).
```

Running this results in the answer set `{ vehicle("scooter"), vehicle("car"), slow_moving("scooter"), highway_possible("car") }`. Given these facts and rules, `highway_possible(car)` can be deduced, indicating the car is highway possible and, due to default negation, the scooter is not highway possible.

This restricted ASP subset can now be formalized. A *Complexity Cat-*

egory specifies the size and shape of the problem of how many predicates and terms exist, how many facts are given, and how large the rules may be. Together with the input facts and a target answer set, it defines a concrete benchmark that both LAMP and ILASP can solve.

Previous work on LLM based logic rule induction has characterized complexity by structural rule graph topology such as Chain, Rooted Directed Graph, Disjunctive, and Mixed categories [10]. Since this work is concerned with correct answer set deduction and rule optimization rather than rule graph structure, complexity is instead quantified by six numerical features. These include number of possible predicates, terms, facts, rules, positive body literals per rule, and negative body literals per rule. All generated programs are also limited to having exactly one answer set.

Definition 1 (Complexity Category)**.** *A* Complexity Category *is a tuple*

$$\mathcal{C} \;=\; \langle\, \mathcal{D},\; \mathcal{T},\; N_F,\; R,\; L^{+},\; L^{-},\; A_{\min} \,\rangle$$

where:

- $\mathcal{D}$ *is a finite set of* descriptors *(predicate symbols);*
- $\mathcal{T}$ *is a finite set of* objects *(constant symbols);*
- $N_F \in \mathbb{N}$ *is the number of* instance facts*;*
- $R \in \mathbb{N}$ *is the maximum number of* rules*;*
- $L^{+} \in \mathbb{N}$ *is the maximum number of* positive literals *in each rule body;*
- $L^{-} \in \mathbb{N}$ *is the maximum number of* negative literals *in each rule body; and*
- $A_{\min} \in \mathbb{N}$ *is the* minimum derived atoms*, the minimum number of atoms that must appear in the answer set beyond the given input facts.*

From a Complexity Category we can define a concrete input instance. Each instance pairs a set of ground facts and a target answer set with a bounded schema that defines the hypothesis space both LAMP and ILASP search over.

Definition 2 (Input Instance)**.** *Given a Complexity Category stated as* $\mathcal{C} = \langle\, \mathcal{D},\; \mathcal{T},\; N_F,\; R,\; L^{+},\; L^{-},\; A_{\min} \,\rangle$ *(Definition 1), an* instance *of* $\mathcal{C}$ *is a tuple*

$$\mathcal{M} \;=\; \langle\, F,\; A^{*},\; \Phi \,\rangle$$

where:

- F *is the* instance facts*, a set of* N_F *ground atoms of the form* $d(o)$ *with descriptor* $d \in \mathcal{D}$ *and object* $o \in \mathcal{T}$ *(e.g.* `d0(o0)`*);*

- A^* *is the* target answer set*, the desired unique stable model that any correct solution* Π *must produce; it must satisfy* $F \subseteq A^*$ *and* $|A^* \setminus F| \geq A_{\min}$*; and*
- Φ *is the* candidate rule schema*, the full set of concrete ASP rules defining the hypothesis space from which* Π *is drawn. Each rule in* Φ *has the form*

$$d_h(X) \text{ :- } p_1(X),\ \ldots,\ p_k(X),\ \texttt{not}\ n_1(X),\ \ldots,\ \texttt{not}\ n_m(X).$$

where $d_h, p_i, n_j \in \mathcal{D}$*,* X *is a single universally-quantified variable,* $k \leq L^+$ *positive body literals, and* $m \leq L^-$ *negative body literals (bounds from* $\mathcal{C}$*, Definition 1).*

A candidate program Π *is any subset* $\Pi \subseteq \Phi$ *with* $|\Pi| \leq R$*; every rule in* Π *is therefore unary, uses only predicates from* $\mathcal{D}$*, and respects the bounds* L^+ *and* L^-*.* Π *is a* solution *to* $\mathcal{M}$ *if and only if*

$$AS(F \cup \Pi) \ = \ \{\, A^* \,\}$$

Meaning the unique answer set of $F \cup \Pi$ *is exactly* A^**.*

The candidate rule schema Φ corresponds directly to the mode bias of the associated ILASP learning task generated for each benchmark.

3.2 Dataset Generation

To instantiate benchmarks across the complexity categories defined above, synthetic data is generated by introducing new possible predicates and terms as needed. To keep the programs sufficiently complex, but manageable by the LLMs, only one stable model is expected and at least one atom must be deduced. The vehicle example above would be represented as shown below.

Listing 2: Benchmark ASP program.

```
d0(o0).
d0(o1).
d1(o1).
d2(X) :- d0(X), not d1(X).
```

This results in the answer set `{ d0(o0), d0(o1), d1(o1), d2(o0) }`, corresponding to the vehicle example answer set.

From each benchmark instance $\mathcal{M}$, a corresponding ILASP task file is generated. The predicates that appear in derived atoms $A^* \setminus F$ are declared as `#modeh` atoms, and all predicates in $\mathcal{D}$ are declared as `#modeb` atoms in both positive and negative forms. Since each benchmark has exactly one answer set, the derived predicates are uniquely determined by A^*, making the `#modeh` declarations both necessary and sufficient. No valid solution can

require a head predicate not already present in $A^* \setminus F$. The target answer set A^* is encoded as a single context-dependent `#pos` example with the instance facts F as context. The inclusion set contains the derived atoms $A^* \setminus F$, the exclusion set contains atoms formed by applying each derived predicate to all terms not present in A^*, and the context contains the instance facts. The corresponding ILASP task for the vehicle example above is shown below.

Listing 3: Corresponding ILASP task.

```
#constant(obj, o0).
#constant(obj, o1).

#modeh(d2(var(obj))).

#modeb(1, d0(var(obj))).
#modeb(1, d0(var(obj)), (negative)).
#modeb(1, d1(var(obj))).
#modeb(1, d1(var(obj)), (negative)).
#modeb(1, d2(var(obj))).
#modeb(1, d2(var(obj)), (negative)).

#pos(eg1, {
  d2(o0)
}, {
  d2(o1)
}, {
  d0(o0).
  d0(o1).
  d1(o1).
}).
```

This gives ILASP the same hypothesis space over which LAMP operates. For benchmarks with a sufficiently small hypothesis space, a search space file can be generated by running ILASP in enumeration mode.

3.3 LLM Prompting

LLM prompting has been shown to be effective in writing ASP programs in such areas as logic puzzle solving [21] and robotic task planning [22]. LLM and ASP based hybrid systems have even been shown to be effective in improving natural language understanding [24], common sense reasoning [31] and as collaborative conversational agents [30].

The prompt provided to the LLMs provides the instance facts (ground atoms), such as `descriptor_0(object_0)`, expected ASP output format `predicate_0(X) :- predicate_1(X), [not] predicate_2(X)...`, and the expected stable model, such as `d0(o0), d0(o1), ...`. The output is then parsed into ASP and run with the ASP solver Clingo. This can then be

passed back if incorrect. This interaction can be represented as shown below.

Definition 3 (LAMP Interaction Protocol)**.** *A* LAMP Interaction Sequence *for instance* $\mathcal{M} = \langle F, A^*, \Phi \rangle$ *is a sequence of pairs*

$$\big\langle\, (\Pi_0, \delta_0),\ (\Pi_1, \delta_1),\ \ldots,\ (\Pi_n, \delta_n) \,\big\rangle$$

where each Π_i *is a candidate program generated by the LLM given* F*,* A^**, and all prior feedback, and* δ_i *is the* feedback signal *produced by running* $F \cup \Pi_i$ *through Clingo:*

$$\delta_i \;=\; \begin{cases} \texttt{correct} & \text{if } AS(F \cup \Pi_i) = \{A^*\}, \\ \texttt{unsatisfiable} & \text{if } AS(F \cup \Pi_i) = \emptyset, \\ \big\langle\, AS(F \cup \Pi_i),\ A^* \,\big\rangle & \text{otherwise.} \end{cases}$$

The sequence terminates when $\delta_n =$ `correct` *or a maximum iteration count is reached. The final output* Π_n *is accepted as the solution if it satisfies* $\mathcal{M}$ *(Definition 2).*

3.4 Evaluation

LAMP and ILASP systems are evaluated by correctness of answer set, time to solve and optimality of ASP programs. The ASP programs generated are evaluated after being run via Clingo. The correctness, number of rules, complexity and how close it is to the expected stable model can then be assessed. This is similar to the process LLM-ARC employs as an actor-critic method, where the LLM Actor generates ASP, while the automated reasoning critic evaluates the code [16].

Other systems use ASP as a scaffolding for robust reasoning [17]. The prompt also instructs the LLM to prefer the minimal rule set with fewest rules and literals, directly encouraging concise encodings.

Beyond correctness, the systems are evaluated on whether the LLM has produced a more concise encoding than the original synthetic program, formalized as follows.

Definition 4 (Program Conciseness)**.** *Given a solution* Π *to instance* $\mathcal{M}$*, its* literal count *is*

$$L(\Pi) \;=\; \sum_{r \in \Pi} \big(|pos(r)| + |neg(r)|\big),$$

where $pos(r)$ *and* $neg(r)$ *are the positive and negative body literals of rule* r*, and* $L(\Pi)$ *is the total number of body literals across all rules. A solution* Π *is* more concise *than* Π' *if* $L(\Pi) < L(\Pi')$*.*

Many other notions of optimality are applicable to ASP programs, including body literal and rule count, stratification, tightness, total grounded rules, and ASP solving time. This work focuses on literal count as a direct measure of complexity with stratification reported separately.

4 Experiments

Experiments were conducted on the synthetic data with varying complexities as specified in the *Dataset Generation* section. A variety of online and local open weight LLMs were selected for testing to get a broad view of capabilities.

4.1 Experimental Setup

A total of 120 synthetic ASP programs were created with varying features for testing. The LLMs chosen included *gpt-5-mini*, which was run via OpenAI's API platform, as well as open weight models *gpt-oss:20b* and *qwen3-coder:30b* which were run locally using Ollama. All ASP solving and analysis was done using Clingo. ILASP was run on each benchmark with a timeout of 420 seconds. Benchmarks not solved within this limit were recorded as failures. Each LAMP run was allowed a maximum of 3 iterations: one initial generation followed by up to 2 rounds of feedback verified by Clingo. Both systems receive the same input instance $\mathcal{M}$ (Definition 2): the target answer set A^* is provided directly to the LLM in the prompt and encoded as a `#pos` example for ILASP. The selected parameters used for complexity classes are listed in table 1. All combinations were used except where positive and negative literals are constrained to sum to no more than the overall literal count. Overall Literals is a generation constraint on the sum $L^+ + L^-$ and is not a field of Definition 1.

Table 1: Parameters associated with Definition 1

Parameter	**Variations**
Descriptors	[10, 6]
Objects	[10, 6]
Facts	[9, 6]
Rules	[11]
Overall Literals	[3, 2]
Negative Literals	[2, 1, 0]
Min Derived Atoms	[3, 1]

4.2 Results and Discussion

Overall results are shown in table 2. Both *gpt-5-mini* and *gpt-oss:20b* perform very well, deducing the correct ASP rules to get the expected stable model in almost all cases. LAMP using *qwen3-coder:30b* performed significantly worse, reaching only 45.0% final accuracy (54/120), with 30.8% correct on the first attempt. Although *qwen3-coder:30b* is tuned for programming, it may not have been tuned for ASP, resulting in poorer performance.

ILASP achieved 97.5% accuracy (117/120) of which all three failures were timeouts, not reasoning errors, occurring on benchmarks with high negation counts and large numbers of derived atoms. LLMs in LAMP, *gpt-5-mini* and *gpt-oss:20b*, solved 3 and 1 of the benchmarks, respectively, on which ILASP timed out, demonstrating that LLM-guided search can succeed in some instances where ILP search cannot.

The iterative feedback loop contributes meaningfully for all models. First attempt accuracy was 94.2% for *gpt-5-mini* (113/120), 90.0% for *gpt-oss:20b* (108/120), and 30.8% for *qwen3-coder:30b* (37/120). Subsequent Clingo verified feedback iterations recovered 7 additional cases for *gpt-5-mini*, 7 for *gpt-oss:20b*, and 17 for *qwen3-coder:30b*. In all cases where the correct stable model was produced, the generated rules never exactly matched those of the original synthetic program, as multiple rule sets can produce the same answer set. Given the sample size of 120 benchmarks, differences of a few cases between systems should be interpreted with caution, as statistical significance may vary.

Table 2: Comparison of all systems including ILASP baseline. First attempt is the accuracy on the initial generation; final accuracy is after up to 2 feedback iterations. ILASP failures are all timeouts (420 s limit); LLM failures are unresolved after all iterations.

System	First attempt	Final	Failures	Failure type
gpt-5-mini	113/120 (94.2%)	120/120 (100%)	0	—
gpt-oss:20b	108/120 (90.0%)	115/120 (95.8%)	5	Max iterations
qwen3-coder:30b	37/120 (30.8%)	54/120 (45.0%)	66	Max iterations
ILASP	—	117/120 (97.5%)	3	Timeout only

Table 3 details the three benchmarks on which ILASP timed out (benchmark indices 7, 27, and 67). All three have elevated grounded rules and hypothesis space sizes relative to the dataset average (Table 4), indicating a larger search space as the cause of timeout. Benchmark 67 has a notably smaller hypothesis space than 7 and 27 yet still timed out, suggesting grounded rules and negation complexity are sufficient to cause timeout independent of hypothesis space size alone. *gpt-5-mini* solved all three, *gpt-oss:20b* solved benchmark 7 but failed on 27 and 67 while *qwen3-coder:30b*

failed all three.

Table 3: Outcomes for the 3 benchmarks where ILASP timed out (420 s limit). ✓= correct stable model produced; × = failed.

Benchmark	gpt-5-mini	gpt-oss:20b	qwen3-coder:30b	ILASP
7	✓	✓	×	Timed out
27	✓	×	×	Timed out
67	✓	×	×	Timed out

Table 4: Structural profile of the 3 ILASP timeout benchmarks vs. dataset average.

Benchmark	Grounded rules	Hypothesis space	Negative literals
7	27	19906	11
27	20	16960	11
67	27	2206	11
Dataset avg	13.28	3255.22	8.80

Figure 2 shows the breakdown of benchmark outcomes per LLM across the three categories of correct on the first attempt (green), correct only after Clingo verified feedback (gold), and failed across all iterations (red).

Figure 3 shows the per-benchmark wall-clock time distribution for each system as a box plot. *gpt-5-mini* is the fastest LLM with a median of 18.0 and a maximum of 137.0, reflecting the low latency of the API-hosted model. *gpt-oss:20b* is slowest with median 50.8 and max 369.6 seconds due to the larger locally-run model size. ILASP has a highly bimodal distribution. The median is only 0.66 seconds as most benchmarks are trivially solved by exhaustive search, but three benchmarks hit the 420.0 second ceiling, pulling the maximum to that value. *qwen3-coder:30b* shows a near uniform distribution with a median of 40.3 and a maximum near the 420.0 timeout at 417.8 seconds, consistent with the model spending maximum iteration budget on hard cases it ultimately fails to solve. Table 5 gives the aggregate statistics.

Table 6 gives the mean output literal count $L(\Pi)$ (Definition 4) per system at original literal counts of 22 and 33, split by correctness. Correct solutions show a large reduction relative to the originals across all systems, averaging 3–5 literals against originals of 22 or 33. This is consistent with the synthetic benchmark structure, where generated programs may include rules not strictly necessary to produce the target answer set.

Figures 4, 5, 6, and 7 compare the unique literal count of the original ASP programs against that of ILASP and the LLM generated rules for each

Figure 2: Stacked bar chart of benchmark outcomes per model: first-attempt success (green), feedback success (gold), and failed (red).

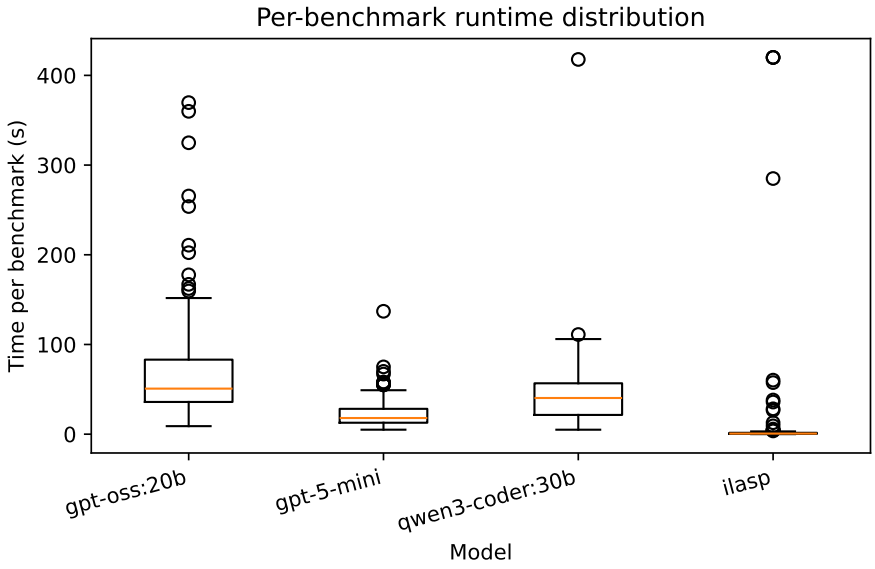

Figure 3: Per-benchmark wall-clock time distribution per system (seconds). Boxes show the interquartile range; whiskers extend to the full range. ILASP failures are capped at the 420 s timeout.

Table 5: Wall-clock time per system (seconds). LLM times accumulate all feedback iterations; ILASP timeouts are recorded as 420 s.

Model	Total (s)	Mean (s)	Median (s)	Max (s)
gpt-5-mini	2837.00	23.64	18.00	137.00
gpt-oss:20b	8579.97	71.50	50.78	369.64
qwen3-coder:30b	5319.65	44.33	40.32	417.77
ILASP	1917.36	15.98	0.66	420.00

Table 6: Mean output literal count $L(\Pi)$ per system for benchmarks with 22 and 33 original body literals, split by correctness. - indicates no data.

Original literals		gpt-5-mini	gpt-oss:20b	qwen3-coder:30b	ILASP
22	Correct	4.81	4.91	4.40	3.67
	Incorrect	–	7.00	4.70	–
33	Correct	3.54	3.96	4.26	2.71
	Incorrect	–	24.00	3.03	–

LLM tested using LAMP. Dot size indicates the number of examples. The left plot shows correct answer sets and the right shows first-attempt failures with recovered cases in gold and failed cases in red.

Figure 4 shows `gpt-5-mini` is generally able to greatly reduce the number of literals needed, in some case reducing number of literals by 6, but in some cases increasing them by 2. LLM `gpt-oss:20b` shown in figure 5 had similar results generally needing even fewer literals, while `qwen3-coder:30b` shown in figure 6 had mixed results. Figure 7 shows ILASP is similarly able to reduce literal counts, though it fails to improve conciseness on the 3 benchmarks where it timed out.

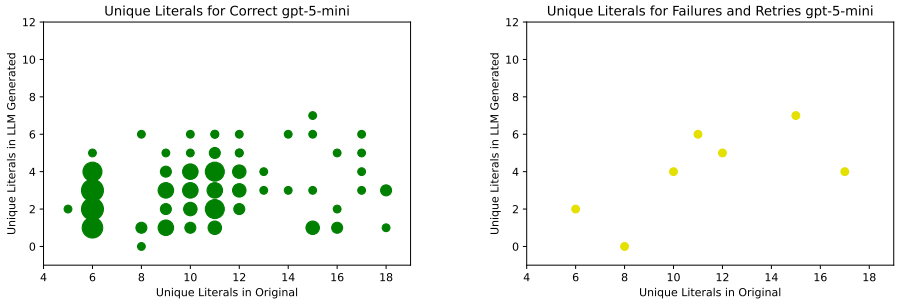

Figure 4: Literal counts for gpt-5-mini (left: correct, right: recovered (gold) / failed (red)).

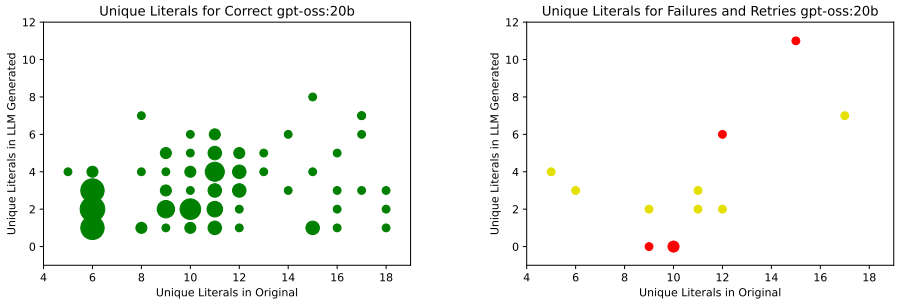

Figure 5: Literal counts for gpt-oss:20b (left: correct, right: recovered (gold) / failed (red)).

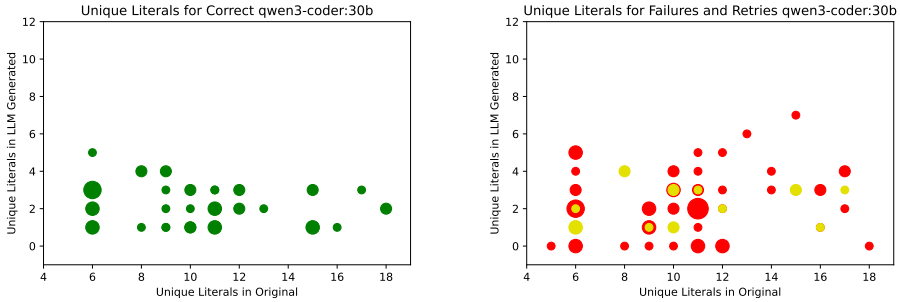

Figure 6: Literal counts for qwen3-coder:30b (left: correct, right: recovered (gold) / failed (red)).

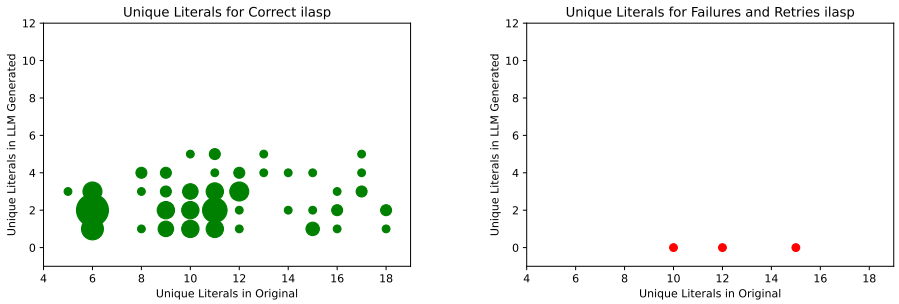

Figure 7: ILASP literal counts (left: correct, right: recovered (gold) / failed (red)).

4.3 Stratification Analysis

An ASP program is *stratified* if no predicate depends negatively on itself (directly or transitively). Stratification is a structural property that simplifies semantics and is often correlated with lower solver complexity. Since the LAMP benchmarks include programs with varying negation depths, it is in-

formative to assess whether LLM-generated programs preserve this property.

Table 7: Stratification of solved programs per system. Original is the 120 benchmark programs.

System	Stratified / Solved
Original	49 / 120
gpt-5-mini	119 / 120
gpt-oss:20b	114 / 115
qwen3-coder:30b	54 / 54
ILASP	117 / 117

Table 7 shows that both ILASP and the LLM-based systems produce programs that are substantially more stratified than the original benchmark programs, of which only 49 of 120 were stratified. ILASP, being a formal inductive learner, produces stratified programs in 117 of its 117 correct solutions. Among the LLMs, *gpt-5-mini* achieves the highest degree of stratification: 119 of its 120 correct programs are stratified, closely followed by *gpt-oss:20b* at 114 of 115. Even *qwen3-coder:30b*, which solved far fewer benchmarks, produced stratified programs in all 54 of its correct solutions. The degree of stratification indicates both learned and generated programs tend toward syntactic simplicity that avoid the cyclic negation present in the synthetic benchmarks.

4.4 Feature Importance

Based on whether the LLM is able to get the correct stable model or not it is possible to deduce which ASP benchmark features are most difficult for LLMs to deduce thereby indicating higher complexity. To do this, syntactic and structural features of the input ASP program, such as number of predicates, facts, rules, and literal counts, are used to predict the likelihood of getting the correct stable model using the tree based machine learning algorithm XGBoost [5], trained on the *qwen3-coder:30b* results. The other two LLMs had too few failures for a meaningful classifier.

From the XGBoost model the importance value of each feature can be calculated. There are multiple ways to get importance, such as gain, the average improvement in the loss function from splits using that feature, or cover, the average number of samples affected by splits using that feature. For this analysis gain is used.

The results can be seen in figure 8. The most important feature is *Solution atoms*, the total number of atoms in the answer set, serving as a proxy for overall problem complexity. The next most important is *Derived atoms*, the number of atoms that must be deduced beyond the input facts, directly reflecting the deductive burden placed on the LLM. *Negative literals* also

ranks highly, consistent with default negation being a known challenge for LLMs. As the XGBoost model is trained on a small sample, these importances should be treated as indicative rather than definitive.

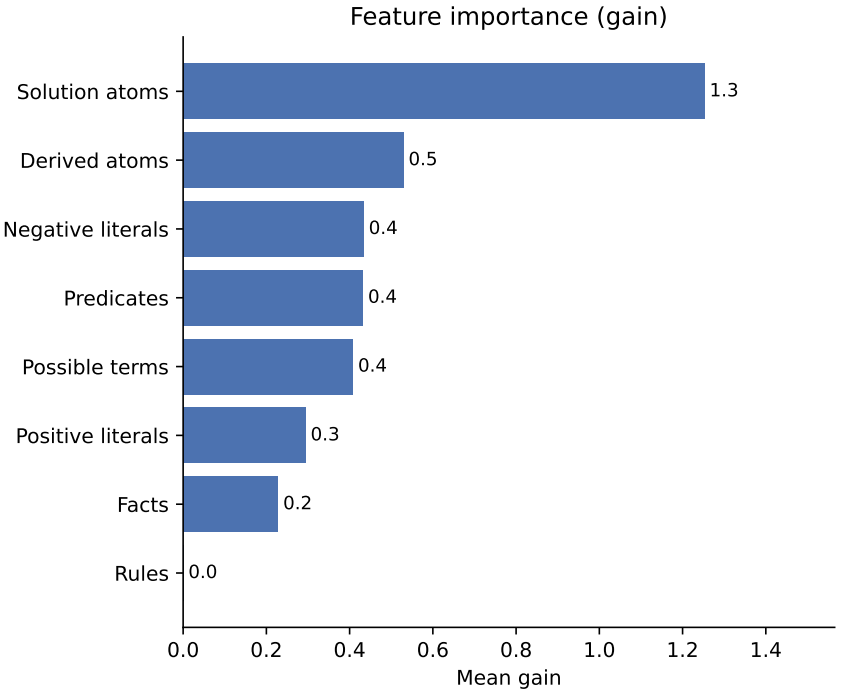

Figure 8: XGBoost feature importance of ASP features predicting LLM success.

5 Related Work

5.1 Inductive ASP Learning and LAS

Recent work has explored the integration of LLMs and ASP with several approaches studying how LLMs can learn from or reason over ASP directly. ILASP [20], the leading ILP system for Learning from Answer Sets was combined with LLMs by Borroto et al. [4] which investigated question answering and inductive learning from answer sets, introducing the LLM2LAS system, which uses LLMs to construct ILASP learning tasks from natural language stories and then runs ILASP to induce the required commonsense rules.

A key distinction from LAMP is that LLM2LAS uses LLMs to *build the ILP task* and then defers to ILASP for search, whereas LAMP uses LLMs to *directly generate* the hypothesis, bypassing exhaustive search. Declarative knowledge distillation techniques have been proposed to transfer structured

reasoning from LLMs into symbolic representations by Eiter et al. (2024) [8].

5.2 Expressivity Graded Evaluation

Gandarela et al. [10] demonstrated that LLMs can perform inductive logic programming, learning logic rules from background knowledge and positive and negative examples when combined with feedback from a formal inference engine such as Prolog. The authors introduced the RuDaS benchmark framework and a systematic methodology for evaluating LLM based theory induction graded by rule expressivity as Chain, RDG, DRDG, and Mixed categories. A key finding from that work is that LLMs are limited by their capacity to track long predicate dependency chains, directly relevant to future LAMP experiments and improvements.

Beyond ASP specific research, logic programming has been used as a tool to evaluate and scaffold LLM reasoning. Hybrid GOFAI LLM systems show how expert systems can be rebuilt using LLM generated logic rules by Garrido et al. (2025) [11], while Xu et al. (2025) [29] shows programming based test evaluators systematically assess LLM reasoning reliability. These works support the use of formal solvers, as in LAMP, to ground and validate LLM outputs.

5.3 LLM Benchmarking on ASP

ASPBench [27] provides benchmarks specifically for grading LLM ASP induction. Ren et al. [25] benchmark 14 LLMs across ASP entailment, verification, and answer set computation tasks, finding that answer set computation remains a significant challenge for LLMs without scaffolding, suggesting that structured prompting and iterative solver feedback, as in LAMP, are likely necessary for reliable ASP induction.

5.4 Planning and NeuroSymbolic Pipelines

Planning provides another closely related line of research. Logical chain of thought tuning for symbolic planning by Verma et al. (2025) [28] and LLM symbolic planning pipelines without expert input by Huang et al. (2024) [14] highlight the benefits of keeping solvers such as ASP planners in the loop. Established ASP planning benchmarks, including Plasp developed by Dimopoulos et al. [7], offer realistic domains that could extend LAMP beyond product configuration. Neuro symbolic refinement frameworks such as PEIRCE further demonstrate how LLMs and formal reasoning systems can iteratively improve each other [23] by Quan et al. (2025).

6 Future Work

Several directions for future work follow naturally. First, iterative prompting could be improved by more explicit chain of thought reasoning, potentially evaluating many candidate rules across multiple instance fact sets per iteration, with Clingo acting as a tool within an LLM based agent loop. The feature sets could also be extended to include greater arity, or additional language features, such as choice rules and head disjunction. Another approach would be from the specific perspective of solving complexity, focusing on solution space features such as, vector cover size, tree depth, feedback vertex set, clique width, tree width, among others [13]. Prompt optimization frameworks such as DSPy [18] could be used to automatically refine prompts and feedback strategies. Even without using LLMs fine-tuned to ASP [6], the models are always improving, so simply updating the models may result in large improvements.

Second, the synthetic datasets could be extended to richer ASP constructs, including higher arity predicates, choice rules, disjunction, and head disjunction, as well as more complex dependency structures such as those proposed by Gandarela et al. (2025) regarding logic program expressivity [10].

Third, evaluation could move beyond synthetic configuration tasks to existing ASP datasets, including ASPBench [27], OOASP models [9], real world ASP encodings, planning benchmarks such as Plasp and PDDL to ASP translations [7], and domains like rail scheduling. Fourth, deeper natural language interfaces to ASP could be explored, covering bidirectional translation between text and ground facts, rules, stable models, and multiple natural languages.

Finally, optimization remains a promising avenue, including other ways of transforming ASP into more efficient ASP, compiling ASP fragments to lower level code, such as C, and tighter integration with solver level optimizations, enabling LLMs to reason not only about correctness but also about grounding and solving performance.

7 Conclusion

This work proposed a structured LLM to ASP Modeling Protocol (LAMP) designed to support the deduction and optimization of ASP rules from instance facts and expected stable models. The results show this as effective for guiding LLMs toward correct and often optimized ASP encodings. By constraining the modeling task and providing iterative feedback via an ASP solver, this system reduces the modeling burden associated with ASP while preserving correctness.

The tested models exhibited clear performance differences, with *gpt-5-*

mini achieving 100% final accuracy (94.2% on first attempt) and *gpt-oss:20b* achieving 95.8% (90%), while *qwen3-coder:30b* reached only 45% (30.8% first attempt) despite its programming focus. These results highlight that strong general code generation ability does not necessarily translate to proficiency in logic programming, underscoring the value of ASP specific evaluation frameworks.

The ILASP baseline achieved 97.5%, with the three failures being search space timeouts rather than reasoning errors. This shows LLMs do not follow the same classical ILP search, they navigate the hypothesis space differently, searching using tokens instead of the hypothesis space, and can succeed where exhaustive search may not. The iterative feedback loop further contributed recovery of many cases for the three LLMs, confirming its value as a component of the LAMP pipeline.

Beyond assisting users in ASP modeling, this protocol provides a system for probing LLM reasoning and logic programming capabilities. Future work includes extending to richer ASP constructs such as disjunction and choice rules, incorporating solver-level performance metrics, and exploring fine-tuned or neuro symbolic hybrids to further improve robustness and scalability.

References

[1] Giovanni Amendola, Francesco Ricca, and Mirek Truszczynski. Beyond np: Quantifying over answer sets, 2019.

[2] Manuel Bichler, Michael Morak, and Stefan Woltran. The power of non-ground rules in answer set programming, 2016.

[3] Bart Bogaerts, Angelos Charalambidis, Giannos Chatziagapis, Babis Kostopoulos, Samuele Pollaci, and Panos Rondogiannis. The stable model semantics for higher-order logic programming, 2024.

[4] Manuel Borroto, Katie Gallagher, Antonio Ielo, Irfan Kareem, Francesco Ricca, and Alessandra Russo. Question answering with llms and learning from answer sets, 2025.

[5] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system, 2016.

[6] Erica Coppolillo, Francesco Calimeri, Giuseppe Manco, Simona Perri, and Francesco Ricca. Llasp: Fine-tuning large language models for answer set programming, 2024.

[7] Yannis Dimopoulos, Martin Gebser, Patrick Lühne, Javier Romero, and Torsten Schaub. plasp 3: Towards effective asp planning, 2018.

[8] Thomas Eiter, Jan Hadl, Nelson Higuera, and Johannes Oetsch. Declarative knowledge distillation from large language models for visual question answering datasets, 2024.

[9] Andreas Falkner, Anna Ryabokon, Gottfried Schenner, and Kostyantyn Shchekotykhin. Ooasp: Connecting object-oriented and logic programming, 2015.

[10] João Pedro Gandarela, Danilo S. Carvalho, and André Freitas. Inductive learning of logical theories with llms: An expressivity-graded analysis, 2025.

[11] Eduardo C. Garrido-Merchán and Cristina Puente. Gofai meets generative ai: Development of expert systems by means of large language models, 2026.

[12] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. Clingo = asp + control: Preliminary report, 2014.

[13] Markus Hecher and Rafael Kiesel. Extended version of: On the structural hardness of answer set programming: Can structure efficiently confine the power of disjunctions?, 2024.

[14] Sukai Huang, Nir Lipovetzky, and Trevor Cohn. Planning in the dark: Llm-symbolic planning pipeline without experts, 2024.

[15] Adam Ishay, Zhun Yang, and Joohyung Lee. Leveraging large language models to generate answer set programs, 2023.

[16] Aditya Kalyanpur, Kailash Karthik Saravanakumar, Victor Barres, Jennifer Chu-Carroll, David Melville, and David Ferrucci. Llm-arc: Enhancing llms with an automated reasoning critic, 2024.

[17] Navdeep Kaur, Lachlan McPheat, Alessandra Russo, Anthony G Cohn, and Pranava Madhyastha. An empirical study of conformal prediction in llm with asp scaffolds for robust reasoning, 2025.

[18] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines, 2023.

[19] Mark Law. Conflict-driven inductive logic programming, 2022.

[20] Mark Law, Alessandra Russo, and Krysia Broda. The ilasp system for inductive learning of answer set programs, 2020.

[21] Naiqi Li, Peiyuan Liu, Zheng Liu, Tao Dai, Yong Jiang, and Shu-Tao Xia. Logic-of-thought: Empowering large language models with logic programs for solving puzzles in natural language, 2025.

[22] Xinrui Lin, Yangfan Wu, Huanyu Yang, Yu Zhang, Yanyong Zhang, and Jianmin Ji. Clmasp: Coupling large language models with answer set programming for robotic task planning, 2024.

[23] Xin Quan, Marco Valentino, Danilo S. Carvalho, Dhairya Dalal, and André Freitas. Peirce: Unifying material and formal reasoning via llm-driven neuro-symbolic refinement, 2025.

[24] Abhiramon Rajasekharan, Yankai Zeng, Parth Padalkar, and Gopal Gupta. Reliable natural language understanding with large language models and answer set programming, 2023.

[25] Lin Ren, Guohui Xiao, Guilin Qi, Yishuai Geng, and Haohan Xue. Can llms solve asp problems? insights from a benchmarking study (extended version), 2025.

[26] Timo Pierre Schrader, Lukas Lange, Tobias Kaminski, Simon Razniewski, and Annemarie Friedrich. A solver-in-the-loop framework for improving llms on answer set programming for logic puzzle solving, 2025.

[27] Stefan Szeider. Asp-bench: From natural language to logic programs, 2026.

[28] Pulkit Verma, Ngoc La, Anthony Favier, Swaroop Mishra, and Julie A. Shah. Teaching llms to plan: Logical chain-of-thought instruction tuning for symbolic planning, 2025.

[29] Zihao Xu, Junchen Ding, Yiling Lou, Kun Zhang, Dong Gong, and Yuekang Li. Socrates or smartypants: Testing logic reasoning capabilities of large language models with logic programming-based test oracles, 2025.

[30] Yankai Zeng and Gopal Gupta. Reliable collaborative conversational agent system based on llms and answer set programming, 2025.

[31] Yankai Zeng, Abhiramon Rajashekharan, Kinjal Basu, Huaduo Wang, Joaquín Arias, and Gopal Gupta. A reliable common-sense reasoning socialbot built using llms and goal-directed asp, 2024.